МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» (ДГТУ)

Факультет
Информатика и вычислительная техника

Кафедра
Программное обеспечение вычислительной техники и автоматизированных систем

Рекуррентные нейронные сети

Практикум

по выполнению лабораторной работы №5
по дисциплине «Нейронные сети и глубокое обучение»

Большие данные и машинное обучение
(указывается направленность (профиль) образовательной программы)

02.03.03 Математическое обеспечение и администрирование информационных систем
(указывается код и наименование направления подготовки)

Ростов-на-Дону
2026 г.

Составители: канд. техн. наук, доц. каф. А.А. Скляренко

УДК 004.432

Многослойная нейронная сеть: метод. указания. – Ростов н/Д: Издательский центр ДГТУ, 2026.

В методическом указании рассматриваются принципы работы с рекуррентными нейронными сетями. Приведены задания к лабораторной работе, помогающие закрепить на практике полученные знания, и контрольные вопросы для самопроверки. Предназначено для обучающихся по направлению 02.03.02 «Математическое обеспечение и администрирование информационных систем», профиль «Большие данные и машинное обучение».

Ответственный за выпуск:
зав. кафедрой «Программное обеспечение вычислительной техники и автоматизированных систем» Долгов В.В.

© А.А. Скляренко,
2026
© Издательский центр ДГТУ, 2026

1 Теоретическая часть

1. Краткая теория

Рекуррентные нейронные сети (Recurrent Neural Networks, RNN) — класс нейронных сетей, специально разработанный для работы с последовательностями данных. В отличие от полносвязных сетей, RNN имеют «память»: они учитывают предыдущие входные данные при обработке текущих.

Ключевая особенность: наличие обратных связей, позволяющих передавать информацию от шага к шагу.

Базовая RNN – слой SimpleRnn в Keras

Улучшенные архитектуры – LSTM, GRU.

```
from tensorflow.keras.layers import LSTM, Dense, GRU, SimpleRNN
```

Пример создания двухслойной рекуррентной нейронной сети

```
# Создание модели
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(X_train.shape[1],
1)))
model.add(LSTM(50, return_sequences=False))
model.add(Dense(25))
model.add(Dense(1))


# Компиляция модели
model.compile(optimizer='adam', loss='mean_squared_error')
```

loss — функция потерь может быть MSE, MAE и др.
Метрики для оценки качества прогноза

- **MSE (Mean Squared Error):**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **RMSE (Root Mean Squared Error):**

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- **MAE (Mean Absolute Error):**

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

2. Исследование временного ряда

Для декомпозиции временного ряда используется библиотека

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

Обращаемся к объекту

```
decomposition = seasonal_decompose(
    df_decomp['Close'],
    model='multiplicative',  # или 'additive'
    period=252
)
```

Возвращаемый объект содержит следующие поля с данными для визуализации: `decomposition.trend` (тренд), `decomposition.seasonal` (сезонность), `decomposition.resid` (остатки).

3. Формирование датасета

Подготовка данных для модели с одномерным входом схематично изображена на рисунке 1.



2 Задание к лабораторной работе

Для всех процессов обучения визуализировать графики и вести лог-файл для сравнения результатов. Обучать модель на основе датасета, указанного в таблице с вариантами.

При выполнении задания в Google Colab необходимо сменить среду выполнения на GPU (в горизонтальном меню Сменить среду выполнения).

Задание 1

Точечный прогноз по одному признаку

1. Проанализировать временной ряд:
    a. Из датасета оставить только один признак, значения которого составляют временной ряд.
    b. определить размер временного интервала (скользящего окна).

    Пример определения размера временного интервала

    Если период записи наблюдения составляет 10 минут, можно убедиться по вышеприведённой таблице. Таким образом, в течение одного часа будет 6 наблюдений. В свою очередь за сутки накапливается 144 (6x24) наблюдения.
    Допустим, необходимо спрогнозировать температуру, которая будет через 6 часов в будущем. Этот прогноз делается на основе имеющихся данных за определённый период: например, за 5 дней наблюдений. Следовательно, для обучения модели необходимо создать временной интервал, ***содержащий последние 720*** *(5x144) наблюдений.*
    c. С помощью Matplotlib визуализировать данные временного ряда.
    d. Выполнить декомпозицию временного ряда, визуализировать тренд, сезонность, остаток.

2. Подготовить данные для обучения сети
    a. Нормализовать данные временного ряда
    b. Создать датасет: разделив временной ряд на последовательности в соответствии с размером временного интервала.
    Примерный фрагмент кода

```
# Создаём последовательности для обучения
def create_sequences(data, seq_length):
    X, y = [], []
```

```
    for i in range(len(data) - seq_length):
        X.append(data[i:i + seq_length])
        y.append(data[i + seq_length])
    return np.array(X), np.array(y)

SEQ_LENGTH = 720  # размер временного интервала
X, y = create_sequences(scaled_data, SEQ_LENGTH)
```

c. Разделить данные на обучение и тестирование
d. Визуализировать (с помощью plot) первую последовательность из набора для обучения x_train[0] и y_train[0] (с помощью scatter).

3. Создать однослойную простую рекуррентную нейронную сеть из 50 нейронов с dropout(0.2) и промежуточным полносвязным слоем на 25 нейронов. Обучить (не менее 50 эпох, а лучше 100), протестировать. Выполнить predict для любой последовательности из тестовых данных и визуализировать саму тестовую последовательность (plot) и два значения y: желаемое и предсказанное (scatter).
4. Преобразовать нейронную сеть из пункта 3, добавив еще один простой рекуррентной слой на 50 нейронов. Повторить задания из 3 пункта, сравнить результаты.
5. Преобразовать нейронную сеть из 4 пункта заменив простые рекуррентные слои на LSTM. Повторить задания из 3 пункта, сравнить результаты.
6. Преобразовать нейронную сеть из 4 пункта заменив простые рекуррентные слои на GRU. Повторить задания из 3 пункта, сравнить результаты.

Задание 2

Интервальный прогноз по одному признаку

1. Временной ряд остается из 1 задания
2. Подготовить данные для обучения сети
    a. Нормализовать данные временного ряда
    b. Создать датасет: разделив временной ряд на последовательности в соответствии с размером временного интервала.

    Примерный фрагмент кода

```
# Создаём последовательности для обучения
def create_sequences_interval(data, seq_length,
forecast_horizon):
    """
    Создаёт последовательности для интервального прогнозирования.

    Параметры:
    - data: нормализованные данные
```

```
    - seq_length: длина входной последовательности (например, 60
дней)
    - forecast_horizon: горизонт прогноза (например, 5 дней)

    Возвращает:
    - X: входные последовательности
    - y: целевые интервалы (массивы значений на горизонте
прогноза)
    """
    X, y = [], []
    for i in range(len(data) - seq_length - forecast_horizon +
1):
        X.append(data[i:i + seq_length])
        y.append(data[i + seq_length:i + seq_length +
forecast_horizon])
    return np.array(X), np.array(y)
```

c. Разделить данные на обучение и тестирование
d. Визуализировать (с помощью plot) первую последовательность из набора для обучения x_train[0] и y_train[0] (с помощью plot).

3. Создать двухслойную рекуррентную нейронную сеть LSTM из 50 нейронов с dropout(0.2) в каждом слое и промежуточным полносвязным слоем на 50 нейронов. Размер последнего полносвязного слоя равен размеру интервала предсказания. Обучить (не менее 50 эпох, а лучше 100), протестировать. Выполнить predict для трех любых последовательностей из тестовых данных и для каждой визуализировать саму тестовую последовательность (plot) и два значения y: желаемые и предсказанные (plot).

Варианты заданий

Если в датасете более 5 классов и много изображений по каждому классу можно сократить до 500 картинок, чтобы уменьшить время обучения.

Вариант	Датасет
1.	**Air Passengers** https://www.kaggle.com/datasets/rakannimer/air-passengers
2.	**Daily Female Births in California** https://www.kaggle.com/datasets/andreicosma/daily-total-female-births-in-california-1959
3.	**Hourly Energy Consumption** https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption
4.	**Huge Stock Market Dataset** https://www.kaggle.com/datasets/borismarjanovic/price-volume-data-for-

	all-us-stocks-etfs
5.	**Global Temperature Time Series** https://www.kaggle.com/datasets/ianpetrustan/global-temperature-time-series
6.	**Retail Sales Dataset** https://www.kaggle.com/datasets/mohammadtalib786/retail-sales-dataset
7.	**Uber Pickups in New York City** https://www.kaggle.com/datasets/fivethirtyeight/uber-pickups-in-new-york-city
8.	**Air Quality Dataset** https://www.kaggle.com/datasets/fedesoriano/air-quality-data-set
9.	**Web Traffic Dataset** https://www.kaggle.com/datasets/raminhuseyn/web-traffic-time-series-dataset
10.	**Bitcoin Historical Dataset** https://www.kaggle.com/datasets/prasoonkottarathil/btcinusd

Можно рассмотреть свой датасет. Он должен представлять собой временной ряд.

3 Материально-техническое обеспечение работы

Аудитория для проведения лабораторных занятий должна быть укомплектована специализированной мебелью и индивидуальными компьютерами следующей минимальной комплектации:

- Процессор: не менее двух исполнительных ядер, совместимый с системой команд x86 и x64, с поддержкой аппаратной виртуализации.
- Оперативная память: не менее 8 Гб.
- Монитор: не менее 24" (дюймов) по диагонали.
- Наличие локальной сети со скоростью обмена не менее 1 Гб/сек.

– Наличие доступа в сеть Интернет со скоростью не менее 1 Мбит/сек.
– Наличие клавиатуры и манипулятора «мышь».

На компьютерах должно быть установлено следующее программное обеспечение:

– Операционная система: Microsoft Windows 10 (или выше) и/или одна из операционных систем семейства Linux, допускающих установку среды исполнения для Python. Допускается конфигурация, когда одна из операционных систем установлена внутри виртуальной машины (гипервизора).
– Среды разработки и исполнения программ: Google Colab, PyCharm, VS Code.

4 Порядок выполнения и сдачи работы

Для выполнения лабораторной работы рекомендуется придерживаться следующего порядка выполнения:

1. Ознакомится с темой и целями лабораторной работы
2. Изучить теоретический материал
3. Подготовить рабочее окружение
4. Разработать программный код лабораторной работы в соответствие с заданием
5. Произвести проверку работоспособности, выполнить тестирование и отладку кода
6. Проанализировать полученные результаты и примененные в ходе решения подходы
7. Выполнить самооценку и рефлексию
8. Сдать лабораторную работу преподавателю и получить от него обратную связь

При сдаче студентом лабораторной работы основным отчетом выступает исходный код самостоятельно созданной в процессе выполнения работы программы.

Исходный код должен быть отформатирован согласно принятым для используемого языка программирования стандартам. Является желательным наличие в исходном коде комментариев, описывающих основные части программы и особенности их функционирования. В то же время студент должен быть готов объяснить работу программы в целом и каждую отдельную ее часть при полном отсутствии комментариев (например, они могут быть удалены преподавателем при сдаче работы).

Обязательным условием сдачи является умение студента самостоятельно восстановить любой участок исходного кода программы (но не более 20 строк подряд) после его удаления. Удаленный участок должен быть самостоятельно набран студентом заново в присутствие преподавателя. При восстановлении участка запрещается использовать операции Undo (Отменить) текстовых редакторов или переписывание кода участка из других источников.

Исходный код сдаваемой программы должен быть представлен в электронном виде.

Сдача исходного кода ранее сдававшихся программ или программ, код которых выложен в сети Интернет, не допускается.

5 Контрольные вопросы к лабораторной работе

1. Что такое рекуррентная нейронная сеть (RNN)? В чём её ключевое отличие от полносвязных и свёрточных сетей?
2. Почему RNN подходят для обработки последовательностей (текстов, временн ых рядов и т. д.)? Приведите 2–3 примера задач.
3. Что означает термин «развёртывание во времени» (unrolling through time) прим енительно к RNN?
4. В чём заключается проблема исчезающего/взрывающегося градиента в RNN? Почему она возникает?
5. Какие методы помогают бороться с исчезающим/взрывающимся градиентом? Кратко опишите каждый.
6. Как LSTM решает проблему долгосрочного запоминания информации? Назови те и кратко охарактеризуйте три основных гейта (вентиля) LSTM.

6 Перечень использованных информационных ресурсов

1. Шолле Франсуа Глубокое обучение на Python. — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»).

2. Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. — СПб.: Питер, 2018. — 480 с.: ил. — (Серия «Библиотека программиста»).

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01. В набор В печать
Объем 0,5 усл.п.л., уч.-изд.л. Офсет. Формат 60х84/16.
Бумага тип №3. Заказ № Тираж 75. Цена

Издательский центр ДГТУ
Адрес университета и полиграфического предприятия:
344010, г. Ростов-на-Дону, пл. Гагарина, 1.